26128 / NGUYỄN VĂN HOÀNG / 20233413

Báo cáo cá nhân tuần 3

Đề tài: Asynchronous virtio-blk and I/O Schedulers on xv6-riscv. Học phần Hệ điều hành ET4290, học kỳ 2026.1. Kỳ kế hoạch tuần 3: 02/11–08/11/2026. Công việc và toàn bộ số liệu thực nghiệm trong báo cáo được thực hiện, kiểm chứng trực tiếp ngày 08/11/2026 trên nhánh `week3/hoang-toolchain-stress` (tách từ `origin/main` `10de5a0`).

Tóm tắt tiến độ cá nhân tuần 3

Trong tuần 3, phần việc độc lập (1 mình phụ trách) của Nguyễn Văn Hoàng tập trung vào hai nhiệm vụ cốt lõi: (1) hoàn thiện khả năng tương thích đa toolchain của bộ khung kiểm thử (`PTO-21 / W3-H01`), loại bỏ sự phụ thuộc vào tên cứng `riscv64-unknown-elf-nm` và chặn việc tạo thư mục kết quả dở dang khi máy thiếu công cụ, đồng thời xác minh lại bản `main` sau khi hợp nhất các nhánh tuần 2–3; và (2) thiết kế, cài đặt bộ kiểm thử stress READ đa tiến trình producer thực (`W3-H02`) dưới cờ biên dịch riêng `-DIO_STRESS_SELFTEST` với kiểm tra dữ liệu đầy đủ `1024 B` trên thiết bị `virtio-blk` thật.

Song song với hai nhiệm vụ lập trình và kiểm thử, Hoàng phụ trách phân tích kỹ thuật và cập nhật trạng thái cho hai câu hỏi `Q1` (commit chuẩn `xv6-riscv` và toolchain) và `Q7` (mở rộng kích thước virtqueue `#define NUM 8` để tăng `max_inflight > 2`) trong issue `PTO-7`, đồng thời thực hiện phản biện kỹ thuật đối với hợp đồng giao diện `W3-G01`, danh mục rà soát vùng nhớ DMA trong `W3-I01` (§10) và tài liệu đề cương `W3-I02`.

Mã công việc	Vai trò	Kết quả kỹ thuật đạt được
PTO-21 / W3-H01	Phụ trách chính (1 mình)	Đồng bộ `NM = $(TOOLPREFIX)nm`, kiểm tra đủ công cụ trước khi tạo thư mục log; xác minh `main` PASS 20/20 boot (`CPUS=1/4`), 4/4 mock, 1 `EXPECTED_FAIL`.
W3-H02	Phụ trách chính (1 mình)	Cài đặt `kernel/io_stress_test.c` và `scripts/check-week03-stress.py`: 4 tiến trình `fork()` × 120 READ = 480 DMA READs/boot; PASS 10/10 boot (4,800 khối 1024 B).
PTO-7 (Q1, Q7)	Phụ trách chính (Q1, Q7)	Chốt phân tích `.xv6-commit = f780d78...` (Q1) và quan hệ `NUM = 8` → `max_inflight = 2` (Q7); giữ nguyên cấu hình trong tuần 3 (`PENDING` phản hồi GVHD).
W3-G01 / I01 / I02	Phản biện kỹ thuật	Rà soát rollback `IO_EAGAIN`, quy tắc không commit `logical_head` sớm, bảng kiểm DMA/ORPHAN §10 và mô tả pipeline driver trong proposal.

Cam kết kiến trúc: Không sửa đổi bất kỳ dòng nào trong 4 tệp lõi runtime (`kernel/io_request.h`, `kernel/io_request.c`, `kernel/io_queue.c`, `kernel/virtio_disk.c`), bảo đảm tương thích 100% với nhánh cài đặt bộ lập lịch SCAN của Phạm Quý Lợi.

KẾT QUẢ THỰC HIỆN TUẦN 3

Đầu vào, trạng thái Repo và Jira

Trạng thái Repository nhóm khi bắt đầu thực hiện

Repository `Asynchronous-virtio-blk-and-IO-Schedulers` được clone mới vào workspace từ GitHub nhóm. Nhánh `origin/main` đang ở commit `10de5a06842e329197f31d882b602dcc2c5300c2` (đã hợp nhất PR #1..PR #6 của tuần 1–2 và PR #7 `week3/hieu-contract-lifecycle-proposal` của Phạm Trung Hiếu). Nhánh làm việc của Hoàng được tách trực tiếp từ `origin/main` với tên `week3/hoang-toolchain-stress`.

Nhánh / PR trên Repo	Commit / Trạng thái	Nội dung liên quan tới phần việc của Hoàng
origin/main	10de5a0 (Merged PR #7)	Chứa lõi async READ tuần 2 và tài liệu hợp đồng/vòng đời/proposal tuần 3 (`docs/week03/`).
week2/hoang-pool-test	e8b9204 (Merged PR #6)	Đầu ra tuần 2 của Hoàng: `virtio_disk_try_submit_read`, `io_hw_test.c`, `check-week02-hoang.py`.
week3/hieu-contract-...	8d298c9 / PR #7	Tài liệu `interface.md` (W3-G01), `lifecycle-review.md` (W3-I01), `proposal.md` (W3-I02).
week3/hoang-toolchain-stress	Working tree trên 10de5a0	Nhánh thực hiện W3-H01 (PTO-21) và W3-H02 của Hoàng trong báo cáo này.

Trạng thái kiểm tra trên Jira (Chỉ đọc, không tự ý chỉnh sửa)

Theo yêu cầu nghiêm ngặt của người dùng, các trang GitHub, Jira (`project = PTO`, `PTO-7?focusedCommentId=10345`) và sơ đồ được kiểm tra ở chế độ chỉ đọc (read-only), không thực hiện bất kỳ thao tác ghi hay thay đổi trạng thái nào trên web.

Issue Jira / Mã kế hoạch	Người phụ trách	Trạng thái ghi nhận & phạm vi của Hoàng
PTO-20 (W3-G01, I01, I02)	Phạm Trung Hiếu	Hiếu đã hoàn thành và merge vào `main`; Hoàng thực hiện phần kiểm chứng chéo bắt buộc.
PTO-21 (W3-H01)	Nguyễn Văn Hoàng	Task cá nhân của Hoàng: sửa harness toolchain (NM, pre-flight check) và xác minh lại `main`.
W3-H02 (Task tuần 3)	Nguyễn Văn Hoàng	Task cá nhân của Hoàng: bổ sung stress READ nhiều producer thực có kiểm dữ liệu 1024 B.
PTO-7 (focusedComment 10345)	Cả nhóm (Hoàng: Q1, Q7)	Theo dõi 7 câu hỏi thiết kế với GVHD; Hoàng phụ trách kỹ thuật mục Q1 và Q7 (PENDING).

KẾT QUẢ THỰC HIỆN TUẦN 3

PTO-21 / W3-H01: Sửa harness toolchain

Vấn đề tồn đọng từ tuần 2 cần giải quyết

Ở tuần 2, script `scripts/check-week02-hoang.py` gọi trực tiếp binary `riscv64-unknown-elf-nm` khi kiểm tra rò rỉ ký hiệu test trong `kernel/kernel` và tạo thư mục `results/...` ngay đầu hàm `main()`. Khi chạy trên môi trường Ubuntu/WSL chỉ cài đặt gói `gcc-riscv64-linux-gnu` (`TOOLPREFIX = riscv64-linux-gnu-`), bước kiểm tra `nm` bị lỗi trừ khi tạo symlink thủ công trong `~/bin`. Ngoài ra, nếu máy thiếu `qemu-system-riscv64` hoặc toolchain, script vẫn tạo thư mục kết quả rỗng gây nhầm lẫn minh chứng.

Các thay đổi kỹ thuật đã thực hiện

1. Trong `Makefile`: bổ sung `NM = $(TOOLPREFIX)nm` đi cùng bộ biến `CC`, `AS`, `LD`, `OBJCOPY`, `OBJDUMP` để đồng bộ hoàn toàn họ công cụ.

2. Trong `mk/project.mk`: thêm target `print-toolprefix` (`@echo $(TOOLPREFIX)`) để script Python truy vấn chính xác tiền tố mà `Makefile` đã tự động dò được; đồng thời cập nhật target `test-week02` sử dụng `$(PYTHON)`.

3. Trong `scripts/check-week02-hoang.py`: xây dựng hàm `resolve_toolprefix()` ưu tiên `--toolprefix` CLI → biến môi trường `TOOLPREFIX` → `make -s print-toolprefix` → tự động dò `riscv64-unknown-elf-`, `riscv64-linux-gnu-`, `riscv64-elf-`.

4. Bổ sung hàm `validate_required_tools(toolprefix)` kiểm tra sự tồn tại của `make`, `qemu-system-riscv64`, `${prefix}gcc`, `${prefix}nm`, `${prefix}objdump`, `${prefix}ld` và hỗ trợ `elf64-big` TRƯỚC khi gọi `outdir.mkdir()`. Nếu thiếu công cụ, thoát ngay với `exit code = 2`.

5. Truyền `TOOLPREFIX=...` nhất quán vào mọi lệnh `make` trong harness và lưu thêm `toolprefix`, `nm_bin`, `gcc_version`, `qemu_version`, `make_version` vào `meta.json` và `summary.md`.

Kiểm chứng trường hợp thiếu công cụ (Negative Pre-flight Check)

```
$ python3 scripts/check-week02-hoang.py --toolprefix riscv64-nonexistent- \
    --results-dir /tmp/no-partial-dir
ERROR: Missing required build/verification tools (TOOLPREFIX='riscv64-nonexistent-'):
  - riscv64-nonexistent-gcc (compiler)
  - riscv64-nonexistent-nm (symbol inspector)
  - riscv64-nonexistent-objdump (disassembler)
  - riscv64-nonexistent-ld (linker)
Exit code: 2 | Directory /tmp/no-partial-dir exists: False
```

KẾT QUẢ THỰC HIỆN TUẦN 3

PTO-21 / W3-H01: Xác minh bản main sau merge

Bước 1: Ghi nhận baseline và kiểm tra nhanh trên main nguyên bản

Trước khi sửa bất kỳ tệp nào, thông tin môi trường và commit gốc (`HEAD = 10de5a06842e329197f31d882b602dcc2c5300c2`, `.xv6-commit = f780d7850765d74cbdcd1fff4b822d8829f7c9bd`, `riscv64-unknown-elf-gcc 13.2.0`, `QEMU 8.2.2`, `GNU Make 4.3`, `Python 3.12.3`) được lưu tại `results/week03/hoang-ubuntu/pre-edit-env.txt`. Lệnh kiểm tra nhanh `--repeat 1 --hw-only` trên `main` nguyên bản đạt PASS (2/2 lượt boot `CPUS=1` và `CPUS=4`, 1 đối chứng âm `EXPECTED_FAIL`).

Bước 2: Kết quả chạy xác minh đầy đủ repeat = 10 (h01-repeat10)

Hạng mục kiểm tra trong h01-repeat10	Cấu hình	Kết quả	Chi tiết quan sát từ log & meta.json
Hardware READ thật (CPUS=1)	10 lần boot lặp	PASS (10/10)	Đạt đủ 15/15 nhóm kịch bản/boot; `submitted=83, completed=81, errors=2, released=83, free_req=16, free_desc=8`.
Hardware READ thật (CPUS=4)	10 lần boot lặp	PASS (10/10)	Đạt đủ 15/15 nhóm kịch bản/boot; `eagain_desc=18..26`, `head_reuse=42..43`, `max_inflight=2`.
Đối chứng âm (negative-corrupt-fixture)	CPUS=1 (1 boot)	EXPECTED_FAIL	Đảo 1 byte fixture trên bản sao `fs.img`; kernel báo đúng `FAIL: T01: 1024-byte pattern`.
Mock selftest: Pool + Wait + Timer	CPUS=1 & CPUS=4	PASS (2/2)	Chạy riêng dưới cờ `-DIO_SELFTEST`; tách bạch với bằng chứng DMA phần cứng thật.
Mock selftest: FCFS Queue + Host Oracle	CPUS=1 & CPUS=4	PASS (2/2)	Chạy riêng dưới cờ `-DIOQ_SELFTEST`; thứ tự phục vụ khớp 100% với oracle trên host.
Khôi phục Production & kiểm tra nm	Production build	PASS	`riscv64-unknown-elf-nm kernel/kernel` xác nhận 0 ký hiệu test; smoke boot lên `init: starting sh`.

Ghi chú trung thực: `20/20` lượt boot phần cứng là `10` lần lặp trên `CPUS=1` và `10` lần lặp trên `CPUS=4` để kiểm tra tính ổn định đồng bộ và ngắt trên từng cấu hình; mỗi lượt boot kiểm tra 15 nhóm kịch bản (`T01..T13`, `ARGS`, `ERROR`), không cộng gộp thành 300 bài test độc lập.

```
Thư mục minh chứng W3-H01:
  results/week03/hoang-ubuntu/pre-edit-env.txt
  results/week03/hoang-ubuntu/main-repeat1-hwonly/{meta.json, summary.md}
  results/week03/hoang-ubuntu/h01-repeat10/{meta.json, summary.md, hw-cpus1-01..10.log, hw-cpus4-01..10.log}
```

KẾT QUẢ THỰC HIỆN TUẦN 3

W3-H02: Thiết kế stress READ đa producer

Khoảng trống của bài test tuần 2 và mục tiêu của W3-H02

Trong tuần 2, bài test phần cứng (`kernel/io_hw_test.c`) phát các yêu cầu đọc bất đồng bộ từ một tiến trình duy nhất; còn bài test 4 tiến trình trong `kernel/io_request_test.c` chỉ dùng `sleep()` trên kênh giả để kiểm tra `kill()/ORPHAN`. Nhiệm vụ `W3-H02` yêu cầu xây dựng một bài kiểm thử stress READ nhiều producer thực sự: nhiều tiến trình con (`fork()`) cùng lúc gọi xuống kernel để phát yêu cầu đọc qua `virtio-blk` DMA thật và đối chiếu toàn bộ `1024 B` của từng khối dữ liệu.

Kiến trúc cài đặt trong kernel/io_stress_test.c (Cờ -DIO_STRESS_SELFTEST)

1. Cô lập hoàn toàn khỏi mã lõi: Mô-đun mới `kernel/io_stress_test.c` chỉ được biên dịch khi `CFLAGS_EXTRA` có `-DIO_STRESS_SELFTEST`. Không sửa đổi `io_request.h/c`, `io_queue.c` hay `virtio_disk.c`.

2. An toàn vùng nhớ DMA (Tuân thủ §10 `W3-I01`): Sử dụng mảng vùng đệm staging tĩnh trong BSS `stress_worker_buf[4][8][1024]` (căn chỉnh 8 byte), tuyệt đối không đặt buffer DMA trên kernel stack của tiến trình.

3. Fixture 8 khối (`blockno = 700..707`) có mẫu byte phụ thuộc cả chỉ số khối `slot` và vị trí byte `off` (`0..1023`): `expected_fixture_byte(slot, off) = ((slot*53 + 19) ^ ((off*97 + (off>>3)*13) & 0xff) ^ (slot << 4)) & 0xff`. Bất kỳ sự trộn lẫn vùng đệm nào giữa các worker hoặc đọc lệch block đều bị phát hiện ngay.

4. Bốn tiến trình producer thực sự (`fork()` trong `user/init.c`): Tiến trình cha gọi `io_selftest(-1)` để khởi tạo và kiểm tra 8 khối fixture, sau đó `fork()` 4 tiến trình con gọi `io_selftest(w)` (`w = 0..3`).

5. Rào chắn đồng bộ và ép tải tài nguyên: Mỗi worker chạy 15 vòng × 8 khối = 120 DMA READs (480 DMA READs/boot). Đầu mỗi vòng, cả 4 worker đồng bộ tại `stress_round_barrier` để cùng đẩy 32 request vào pool 16 slot và thiết bị `max_inflight = 2`. Trong 14 vòng đầu, worker giữ các slot `DONE` tới cuối đợt mới gọi `io_req_release()`, ép hệ thống rơi vào trạng thái cạn pool (`eagain_pool > 0`) và hết chuỗi 3 descriptor (`eagain_desc > 0` trên `CPUS=4`).

6. Kiểm tra thu hồi toàn cục (`io_selftest(-2)`): Sau khi `wait()` đủ 4 tiến trình con, tiến trình cha xác nhận `verified_bytes == 491520`, `mismatch == 0`, `free_req == 16`, `free_desc == 8`, `queued == inflight == done == error == 0`, và `submitted == completed == released == 480`.

KẾT QUẢ THỰC HIỆN TUẦN 3

W3-H02: Kết quả thực nghiệm và đối chứng âm

Kết quả chạy 5 lần lặp trên CPUS=1 và 5 lần lặp trên CPUS=4 (Tổng 4,800 DMA READs)

Chỉ số kiểm chứng (mỗi lần boot)	Kết quả CPUS=1 (5 boots)	Kết quả CPUS=4 (5 boots)	Diễn giải kỹ thuật
Số tiến trình producer (`fork()`)	4 (`pid = 2, 3, 4, 5`)	4 (`pid = 2, 3, 4, 5`)	4 tiến trình user độc lập chạy đồng thời.
Số yêu cầu DMA READ hoàn tất	480 / boot (2,400 tổng)	480 / boot (2,400 tổng)	Mỗi worker hoàn tất đủ 15 vòng × 8 = 120 READs.
Tổng số byte kiểm tra từng byte	491,520 B / boot	491,520 B / boot	So khớp đủ 1024/1024 B trên mọi khối đọc về.
Số lỗi sai lệch dữ liệu (`mismatch`)	0	0	Không trộn buffer DMA, không hoàn tất sớm.
Số lần cạn pool (`eagain_pool`)	116 lần / boot	234 lần / boot	Chạm giới hạn `N_IO_REQ = 16`, thoái lui và thử lại an toàn.
Số lần thiếu 3 descriptor (`eagain_desc`)	0 lần / boot	116 lần / boot	Trên CPUS=4, 4 hart tranh chấp chạm `max_inflight=2` (`IO_EAGAIN`).
Số lần tái sử dụng head (`head_reuse`)	451 lần / boot	433 – 438 lần / boot	Chuỗi 3 descriptor được IRQ thu hồi và cấp phát lại an toàn.
Trạng thái sau khi drain (`op = -2`)	`free_req=16, free_desc=8`	`free_req=16, free_desc=8`	`queued=inflight=done=error=0; submitted=completed=released=480`.

Đối chứng âm (negative-corrupt-stress-fixture) và khôi phục Production

Để chứng minh bộ kiểm tra 1024 B thực sự đọc dữ liệu từ đĩa qua DMA chứ không đọc vùng nhớ đệm sẵn có, harness tạo bản sao `fs.img` và đảo đúng `1 byte` (`^= 0xFF`) tại `blockno = 703` (`slot = 3`), `offset = 511`. Khi boot, kernel phát hiện ngay ở bước kiểm tra mẫu 1024 B và in `[IO_STRESS:FAIL] prep_read_data (703 != 703): 1024-byte fixture pattern`, được ghi nhận là EXPECTED_FAIL.

```
[IO_STRESS:PASS] worker w=0 pid=2 blocks=120 bytes=122880 alloc_retry=36 submit_retry=52 mismatch=0
[IO_STRESS:PASS] worker w=1 pid=3 blocks=120 bytes=122880 alloc_retry=60 submit_retry=60 mismatch=0
[IO_STRESS:PASS] worker w=2 pid=4 blocks=120 bytes=122880 alloc_retry=68 submit_retry=68 mismatch=0
[IO_STRESS:PASS] worker w=3 pid=5 blocks=120 bytes=122880 alloc_retry=70 submit_retry=70 mismatch=0
[IO_STRESS:INFO] summary workers_ok=4/4 total_blocks=480 verified_bytes=491520 mismatch=0
  free_req=16 free_desc=8 submitted=480 completed=480 released=480
  eagain_pool=234 eagain_desc=116 head_reuse=435
IO-STRESS-USER: PASS (workers=4 blocks=480 bytes=491520)
```

KẾT QUẢ THỰC HIỆN TUẦN 3

PTO-7: Phân tích kỹ thuật Q1 và Q7

Trong issue PTO-7 (chú ý chuỗi trao đổi tại focusedCommentId=10345), nhóm tổng hợp 7 câu hỏi thiết kế và phạm vi kỹ thuật cần chốt với GVHD. Nguyễn Văn Hoàng phụ trách trực tiếp phần phân tích và thực thi kỹ thuật của hai mục Q1 và Q7.

PTO-7 / Q1: Chốt commit/tag chuẩn của xv6-riscv và bộ công cụ biên dịch

Khía cạnh	Thực trạng trong Repository & Phân tích của Hoàng	Quyết định áp dụng trong Tuần 3
Mã nguồn nền xv6-riscv	Ghim tại .xv6-commit = f780d7850765d74cbdcd1fff4b822d8829f7c9bd (upstream mit-pdos/xv6-riscv ngày 15/10/2026). Bản này đã có sẵn cờ -fno-delete-null-pointer-checks và nhóm đã bổ sung sleep_prepare() trong kernel/proc.c.	Giữ nguyên commit f780d78... theo đúng ràng buộc của W3-H01 để bảo toàn tính tái lập. Trạng thái GVHD: PENDING.
Bộ công cụ (Toolchain)	Các thành viên trong nhóm dùng cả riscv64-unknown-elf- (GCC 13.2.0) và riscv64-linux-gnu-. Nhiệm vụ W3-H01 đã hợp nhất cơ chế dò TOOLPREFIX và NM = $(TOOLPREFIX)nm.	Hỗ trợ tự động cả hai họ toolchain; không cần symlink thủ công trong ~/bin.

PTO-7 / Q7: Khả năng mở rộng kích thước virtqueue (#define NUM 8) để tăng max_inflight > 2

Trong kernel/virtio.h, hằng số #define NUM 8 quy định tổng số descriptor của virtqueue. Trong kernel/virtio_disk.c, mỗi yêu cầu I/O khối sử dụng hàm alloc3_desc(idx) để cấp phát chuỗi 3 descriptor liên kết (VRING_DESC_F_NEXT): desc[idx[0]] chứa header 16 byte (type, reserved, sector), desc[idx[1]] trỏ tới vùng đệm dữ liệu 1024 byte, và desc[idx[2]] trỏ tới byte trạng thái 1 byte. Do đó, số yêu cầu tối đa có thể nằm trên virtqueue cùng lúc là floor(8 / 3) = 2 (max_inflight = 2), còn dư 2 descriptor lẻ không đủ lập chuỗi thứ ba.

Đánh giá khả năng mở rộng: Toàn bộ các cấu trúc trong virtio_disk.c (disk.desc, disk.avail, disk.used, disk.free[NUM], disk.info[NUM], disk.last_async[NUM], disk.ops[NUM]) đều được tham số hóa theo NUM. Với giao thức Virtio MMIO legacy (v1) trên QEMU, thanh ghi VIRTIO_MMIO_QUEUE_NUM_MAX cho phép đặt NUM là lũy thừa của 2 như 16 (max_inflight = 5) hoặc 32 (max_inflight = 10) mà toàn bộ hai vòng avail/used vẫn nằm gọn trong một trang 4096 B.

Quyết định trong tuần 3: Ngay cả khi NUM = 8 (max_inflight = 2), với N_IO_REQ = 16, khi có nhiều tiến trình phát I/O đồng thời (như trong bài stress W3-H02), có tới 14 request xếp hàng trong io_queue. Độ sâu hàng đợi này hoàn toàn đủ để bộ lập lịch SCAN lựa chọn thứ tự phục vụ khác biệt rõ rệt so với FIFO. Vì vậy, theo quyết định S4/D20 trong docs/week03/interface.md, nhóm giữ nguyên NUM = 8 trong tuần 3 trong khi chờ ý kiến phản hồi của GVHD trên PTO-7 / Q7 (PENDING).

KẾT QUẢ THỰC HIỆN TUẦN 3

Phản biện kỹ thuật W3-G01, W3-I01, W3-I02

Theo phân công tuần 3, bên cạnh hai task độc lập W3-H01 và W3-H02, Hoàng thực hiện rà soát và kiểm chứng kỹ thuật đối với ba tài liệu nền tảng do Hiếu biên soạn trên nhánh main (docs/week03/interface.md, lifecycle-review.md, proposal.md).

1. Kiểm chứng hợp đồng giao diện W3-G01 (docs/week03/interface.md)

Đã đối chiếu mã thực tế của virtio_disk_try_submit_read(struct io_req *r) trong kernel/virtio_disk.c với hợp đồng W3-G01: (a) Khi alloc3_desc() thất bại, hàm giải phóng ngay các descriptor vừa lấy dở (nếu có), tăng disk.stats.eagain_desc++, giữ nguyên r->state == IO_REQ_QUEUED và trả về IO_EAGAIN (-11) dưới cùng khóa io_lock. (b) Trong vòng lặp dispatch, io_queue_select_next_locked() chỉ chọn ứng viên và gỡ tạm khỏi hàng đợi; nếu try_submit trả IO_EAGAIN, io_queue_requeue_front_locked() đặt lại request về hàng đợi và tuyệt đối chưa cập nhật logical_head, direction hay ghi nhận thứ tự dispatch.

2. Kiểm chứng danh mục an toàn vùng nhớ DMA & ORPHAN trong W3-I01 (§10 lifecycle-review.md)

Mục kiểm tra trong §10.1	Cơ chế bảo vệ trong mã nguồn thực tế	Kết quả rà soát
Không giải phóng sớm slot đang DMA	io_req_release() kiểm tra trạng thái dưới io_lock; gọi panic() nếu state == IO_REQ_QUEUED hoặc IO_REQ_INFLIGHT.	Đạt (Đã kiểm chứng qua T04 & W3-H02)
Không đặt vùng đệm DMA trên stack	Mọi vùng đệm trong io_request_test.c, io_hw_test.c và io_stress_test.c đều khai báo static trong BSS hoặc thuộc buf cache.	Đạt (Không có nguy cơ DMA UAF khi process thoát)
Thu hồi đúng khi tiến trình bị kill()	io_req_wait() gắn cờ IO_REQ_FLAG_ORPHAN: nếu QUEUED thì gỡ khỏi queue và free ngay; nếu INFLIGHT thì chờ IRQ DMA xong mới tự free trong io_req_complete() và tăng released++.	Đạt (Bảo toàn bất biến submitted == released)
Thứ tự khóa io_lock → p->lock	Sử dụng sleep_prepare(r, &io_lock) trước khi nhả io_lock và gọi sched(); io_req_complete() gọi wakeup_RequestWaiters() dưới io_lock.	Đạt (Không mất wakeup, không deadlock)

3. Rà soát đề cương kỹ thuật W3-I02 (docs/week03/proposal.md)

Đã rà soát mục mô tả pipeline driver bất đồng bộ, cách phân biệt IO_HEAD_SYNC và IO_HEAD_ASYNC trên used ring, diễn giải max_inflight = 2 (NUM = 8, 3 descriptor/request) và bảng tổng hợp minh chứng thực nghiệm tuần 2 trong docs/week03/proposal.md. Các số liệu và giới hạn phạm vi (READ-only 1024 B, chưa tích hợp thay bread(), chưa đo tốc độ phần cứng) đều chính xác.

KẾT QUẢ THỰC HIỆN TUẦN 3

Danh mục thay đổi mã nguồn và tái lập

Tổng hợp các tệp thay đổi trên nhánh week3/hoang-toolchain-stress

Bốn tệp lõi thời gian chạy (`kernel/io_request.h`, `kernel/io_request.c`, `kernel/io_queue.c`, `kernel/virtio_disk.c`) có **0 dòng thay đổi**, đảm bảo không phát sinh xung đột khi ghép nhánh với phần cài đặt SCAN của Lợi.

Tệp trong Repository	Loại thay đổi	Nội dung chi tiết
Makefile	Sửa (+7 dòng)	Thêm `NM = $(TOOLPREFIX)nm`; nối `$K/io_stress_test.o` khi có `-DIO_STRESS_SELFTEST`.
mk/project.mk	Sửa (+10/-2 dòng)	Thêm target `print-toolprefix`, dùng `$(PYTHON)` cho `test-week02`, thêm `test-week03-stress`.
scripts/check-week02-hoang.py	Sửa (+198/-62 dòng)	Tự động phân giải TOOLPREFIX/NM; kiểm tra đủ binary trước khi tạo thư mục log (`W3-H01`).
kernel/io_stress_test.c	Thêm mới (480 dòng)	Bài kiểm thử stress READ 4 tiến trình producer × 120 READ = 480 DMA READs/boot, kiểm đủ 1024 B (`W3-H02`).
kernel/io_request_test.c	Sửa (+8 dòng)	Điều phối `sys_io_selftest(op)` sang `io_stress_selftest(op)` dưới `#elif defined(IO_STRESS_SELFTEST)`.
user/init.c	Sửa (+30 dòng)	Nhánh IO_STRESS_SELFTEST: gọi `io_selftest(-1)`, `fork()` 4 worker `0..3`, `wait()` và `io_selftest(-2)`.
scripts/check-week03-stress.py	Thêm mới (630 dòng)	Harness tự động cho `W3-H02`: lặp CPUS=1/4, đối chứng âm lật 1 byte, khôi phục production và kiểm `nm`.
docs/testing.md & report-hoang.md	Sửa / Thêm mới	Tài liệu hóa cách chạy `W3-H01`, `W3-H02` và báo cáo chi tiết tuần 3 của Hoàng.

Lệnh tái lập đầy đủ từ thư mục Repository

```
cd /home/ubuntu/Documents/Codex/2026-10-23/t/outputs/Asynchronous-virtio-blk-and-IO-Schedulers
git status --short && git rev-parse HEAD

# 1. Verify W3-H01 (repeat=10: 20 real virtio-blk boots + negative control + 4 mock + nm check)
python3 scripts/check-week02-hoang.py --repeat 10 \
  --results-dir results/week03/hoang-ubuntu/h01-repeat10

# 2. Verify W3-H02 (repeat=5: 10 boots x 480 DMA READs = 4,800 READs full 1024B check + negative control)
python3 scripts/check-week03-stress.py --repeat 5 \
  --results-dir results/week03/stress/hoang-ubuntu-repeat5

# 3. Text encoding check and fast smoke test on production build
git diff --check && make check-text && make test-fast
```

KẾT QUẢ THỰC HIỆN TUẦN 3

Nội dung cập nhật Jira và trình bày

PTO-21 (W3-H01) — Nội dung chuẩn bị để cập nhật Description / Comment trên Jira

Đã đồng bộ `NM = $(TOOLPREFIX)nm` trong `Makefile`, thêm target `print-toolprefix` trong `mk/project.mk` và cập nhật `scripts/check-week02-hoang.py` để tự động phân giải `TOOLPREFIX` (`riscv64-unknown-elf-` hoặc `riscv64-linux-gnu-`) cũng như kiểm tra đầy đủ công cụ (`make`, `qemu-system-riscv64`, `gcc`, `nm`, `objdump`, `ld`, `elf64-big`) trước khi tạo thư mục kết quả. Đã xác minh trên nhánh `week3/hoang-toolchain-stress` (tách từ `main` `10de5a0`): `20/20` lượt boot phần cứng (`10` `CPUS=1` + `10` `CPUS=4`, 15 kịch bản/boot) PASS, 1 đối chứng âm `EXPECTED_FAIL`, `4/4` bộ test mock PASS, bản production sạch ký hiệu test. Minh chứng: `results/week03/hoang-ubuntu/h01-repeat10/summary.md`.

W3-H02 — Nội dung chuẩn bị để cập nhật báo cáo / Jira

Đã bổ sung `kernel/io_stress_test.c` (dưới cờ `-DIO_STRESS_SELFTEST`) và `scripts/check-week03-stress.py`: sử dụng 4 tiến trình con `fork()` đồng thời phát `15 × 8 = 120` yêu cầu đọc/worker (`480` DMA READs/boot) trên 8 khối fixture (`700..707`) và kiểm tra từng byte trên toàn bộ `1024 B` (`491,520 B`/boot). Chạy `repeat = 5` đạt PASS trên cả `CPUS=1` và `CPUS=4` (`4,800` DMA READs khớp 100%, `eagain_pool = 116..234`, `eagain_desc = 116` trên `CPUS=4`, `head_reuse = 433..451`, `free_req = 16`, `free_desc = 8`). Đối chứng âm lật 1 byte đạt `EXPECTED_FAIL`. Minh chứng: `results/week03/stress/hoang-ubuntu-repeat5/summary.md`.

Các mục còn PENDING (Báo cáo trung thực theo quy ước nhóm)

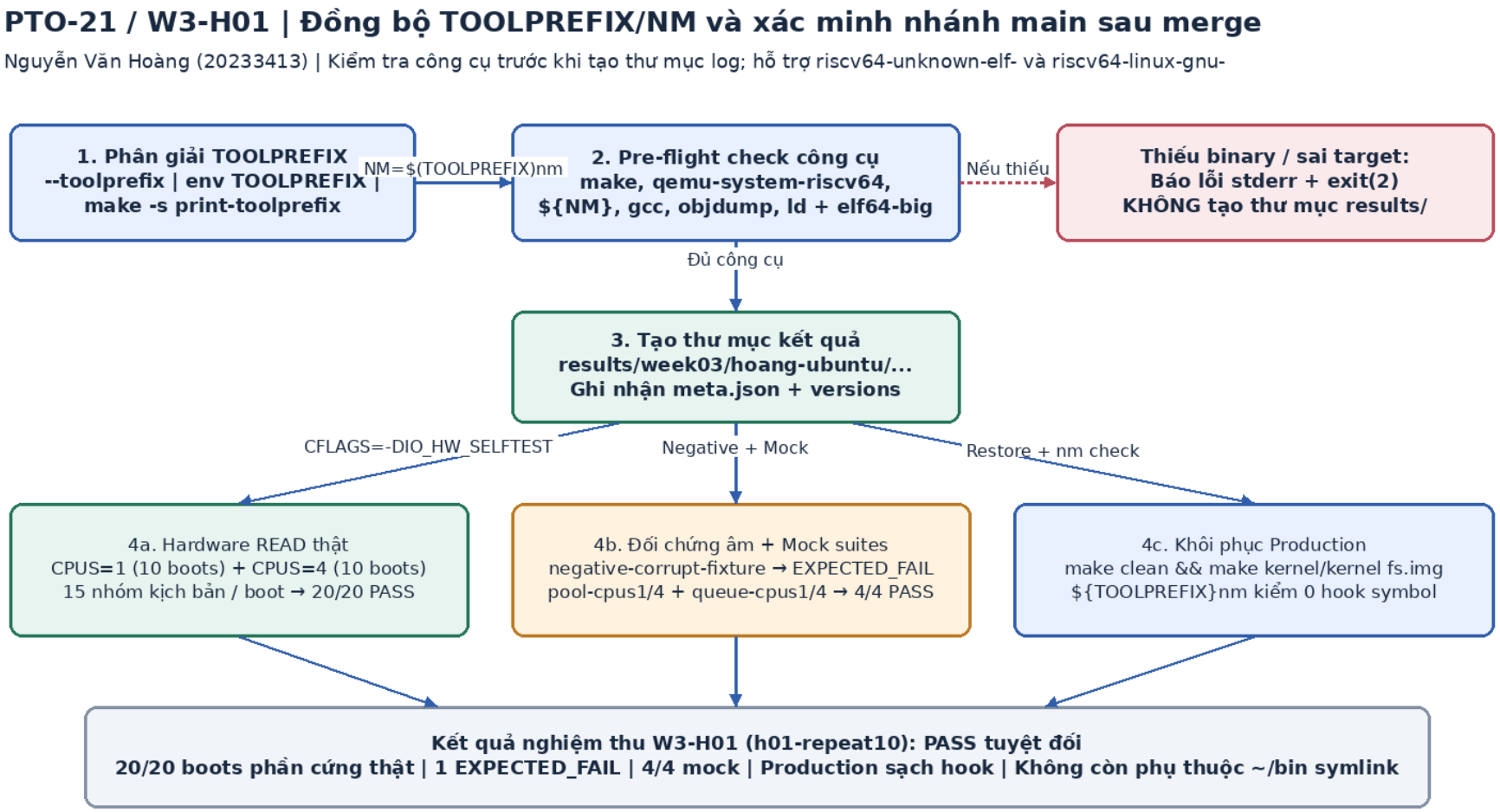
1. Chạy đối chiếu chéo với `TOOLPREFIX=riscv64-linux-gnu-` trên máy của Phạm Quý Lợi: Trên máy Hoàng dùng `riscv64-unknown-elf-` (`GCC 13.2.0`); việc chạy lại `--repeat 1` với `riscv64-linux-gnu-` trên máy Lợi được giữ ở trạng thái `PENDING` chờ Lợi đồng bộ nhánh.
2. Phản hồi của GVHD cho `PTO-7` (`Q1..Q7`): Đang ở trạng thái `PENDING`; trong tuần 3 giữ nguyên `.xv6-commit = f780d78...` và `NUM = 8`.

Kịch bản trình bày cá nhân (5–8 phút)

Phút 0–1: Nêu phạm vi 2 task độc lập (`W3-H01`, `W3-H02`), 2 mục `PTO-7` (`Q1`, `Q7`) và nguyên tắc không chạm 4 tệp lõi runtime. Phút 1–3: Trình bày `W3-H01`, cơ chế kiểm tra công cụ trước khi tạo thư mục log và kết quả `20/20` lượt boot trên `main` sau merge. Phút 3–6: Mở sơ đồ `W3-H02` và log `hoang-ubuntu-repeat5/summary.md`, giải thích cách 4 tiến trình `fork()` tạo áp lực `eagain_pool=234`, `eagain_desc=116` trên `CPUS=4`, kiểm khớp `491,520 B`/boot và đối chứng âm lật 1 byte. Phút cuối: Trình bày phân tích `NUM = 8` (`Q7`) và các mục `PENDING`.

SƠ ĐỒ XUẤT TỪ DRAW.IO

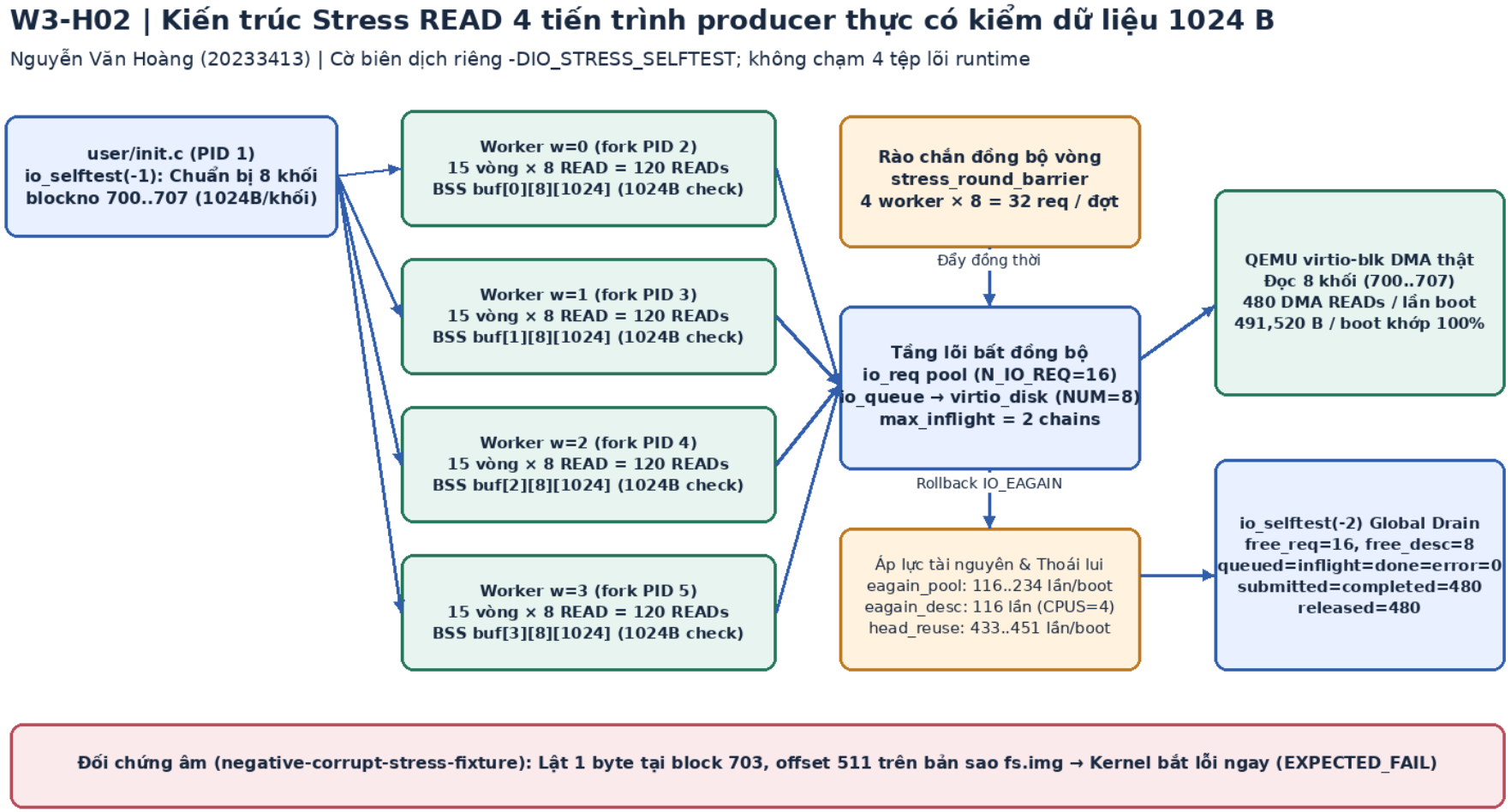
Phụ lục A | Luồng kiểm tra toolchain W3-H01



Trang draw.io 01 (PTO-21 / W3-H01). Phân giải tự động TOOLPREFIX/NM, kiểm tra đủ công cụ trước khi tạo thư mục kết quả và xác minh 20 lượt boot phần cứng.

SƠ ĐỒ XUẤT TỪ DRAW.IO

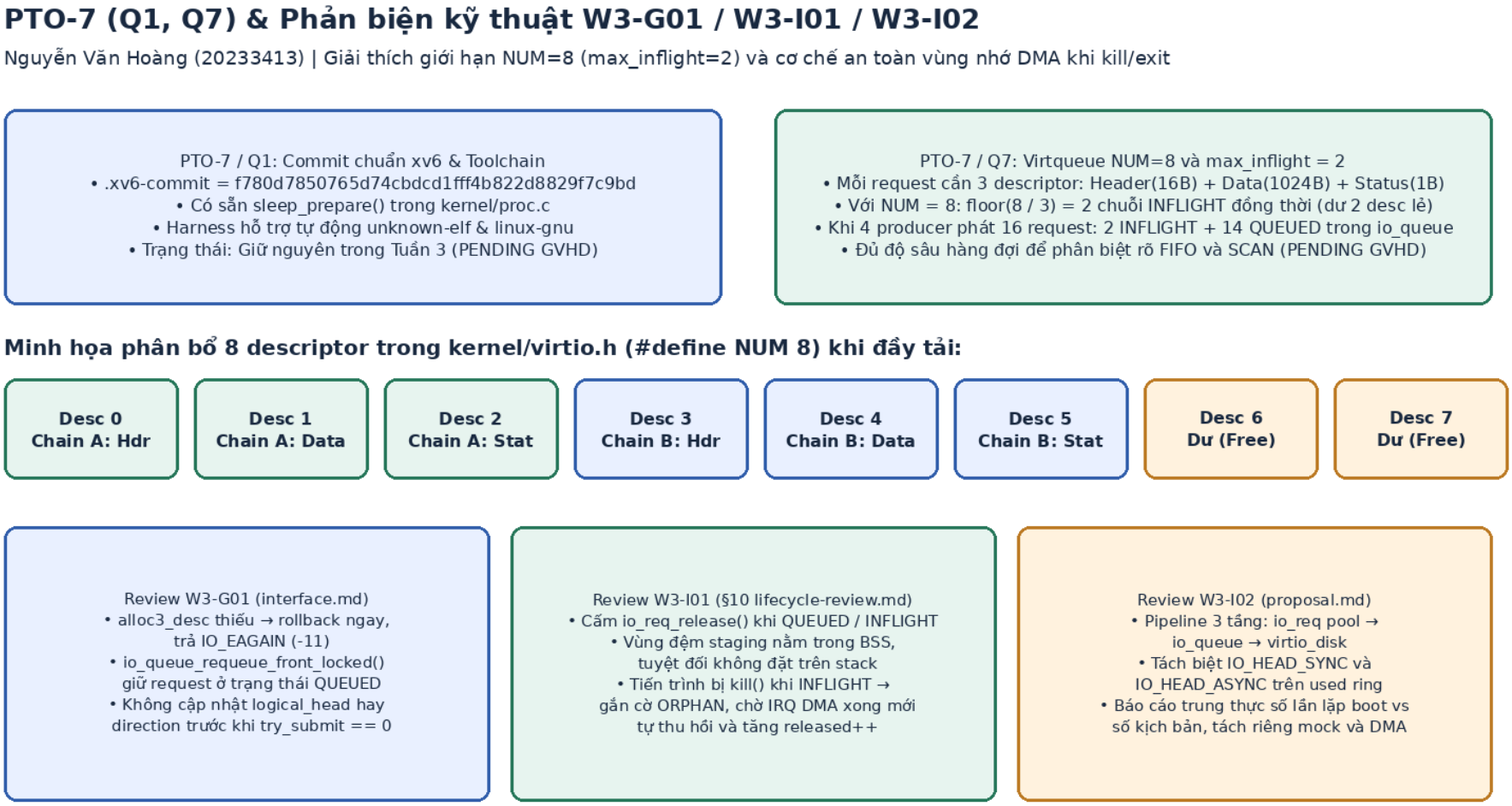
Phụ lục B | Kiến trúc stress READ 4 producer W3-H02



Trang draw.io 02 (W3-H02). Bốn tiến trình con fork() đồng bộ qua rào chắn, phát 480 yêu cầu DMA READ/boot, kiểm đủ 1024 B/khối và đối chứng âm lật 1 byte.

SƠ ĐỒ XUẤT TỪ DRAW.IO

Phụ lục C | Phân tích virtqueue NUM=8 và PTO-7 (Q1, Q7)



Trang draw.io 03 (PTO-7 & Review tuần 3). Minh họa 2 chuỗi 3 descriptor trên virtqueue NUM=8 (max_inflight=2) và các chốt an toàn vùng nhớ DMA / ORPHAN.